FORM-FB-01

Fichas bibliográficas

Asignatura: Sistemas distribuidos

Estudiante: Roger Steven García Sangoquiza

Tema de TA-IND-01. Análisis crítico de Apache Cassandra en comparación con MongoDB, centrado en replicación, consistencia y tolerancia a fallos.

Selección. El artículo 4 aporta evidencia experimental directa sobre los dos motores. Los artículos 1, 2, 3 y 5 aportan contexto sobre colocación de cargas y modelos de fallo, o criterios para diseñar e interpretar las mediciones del informe.

Declaración de uso de inteligencia artificial

Modelos: GPT-6 Sol/GPT-6 Astra.

Entorno: Codex.

Descripcion del uso:

Primero utilice ChatGPT para realizar una búsqueda preliminar de ocho artículos académicos. Revisé manualmente las opciones, comprobé los DOI y su correspondencia con las publicaciones, y confirmé que podía acceder a los textos (PDF) completos. A partir de esa revisión seleccioné los cinco artículos de la tarea y descargué personalmente sus referencias en formatos BibTeX y RIS.

Con esos materiales, ChatGPT me ayudó a unificar las referencias en un archivo '.bib', cotejar los datos bibliográficos con los PDF y preparar una matriz de lectura. La utilicé como guía para preparar mis resúmenes, elegir los datos concretos y redactar mis notas personales.

En una etapa posterior, ChatGPT generó el borrador de la primera ficha con un resumen, dato y nota falsos, de manera que yo pueda tener una visualizacion de la ficha y organizar mejor el documento final. Luego, le proporcione todas las fichas, organizó y corrigió todo el contenido del documento en el main.tex, las referencias en un archivo .bib, preparó las instrucciones de compilación y compiló el PDF con pdfLaTeX y Biber.

Ficha 1. Planificación de cargas en Kubernetes

Referencia IEEE

V. Bregar and M. B. Juric, «Toward adaptive workload scheduling in Kubernetes across the edge-cloud continuum», *Cluster Computing*, vol. 29, no. 15, Oct. 15, 2026, Art. no. 850. DOI: 10.1007/s10586-026-06669-7

DOI

10.1007/s10586-026-06669-7

Resumen

El artículo examina cómo asignar cargas entre recursos de nube (cloud) y niebla (fog) con capacidades y condiciones de red distintas. Proponen CosmoSpan, una plataforma basada en Kubernetes que recoge información de los nodos, interpreta requisitos declarados por las aplicaciones y ajusta la colocación cuando cambia el entorno. Los autores comparan su planificador con el predeterminado de Kubernetes y con Diktyo mediante escenarios emulados. También realizan una validación en un despliegue geográficamente distribuido. CosmoSpan obtiene mejores puntuaciones medias de colocación según la métrica del estudio. El trabajo permite examinar el coste de considerar simultáneamente capacidad, red y disponibilidad al asignar cargas.

Dato concreto y localización

En la sección 7.3, p. 16, CosmoSpan obtiene 98,68 puntos en el indicador compuesto de colocación, frente a 69,99 del planificador de Kubernetes y 77,87 de Diktyo. Son puntuaciones de colocación calculadas por los autores.

Nota personal y uso en TA-IND-01

El artículo permite separar dos decisiones de diseño. La primera es dónde se ejecuta un proceso y la segunda es qué nodos conservan copias de sus datos. Esa separación ayuda a explicar por qué la distribución de Cassandra y MongoDB puede condicionar la latencia observada en un despliegue. CosmoSpan aporta contexto sobre colocación de cargas. Las garantías de cada base deben justificarse con fuentes específicas.

Ficha 2. Consenso, replicación y seguridad

Referencia IEEE

L. Mastromauro, D. Souto Andrade, M. Ozgur Ozmen, and M. A. Kinsy, «Survey of attacks and defenses on consensus algorithms for secure data replication in distributed systems», *IEEE Access*, vol. 13, pp. 143 631–143 667, 2025. DOI: 10.1109/ACCESS.2025.3597260

DOI

10.1109/ACCESS.2025.3597260

Resumen

El artículo estudia los ataques contra algoritmos de consenso usados para replicar datos. Organizan la revisión mediante búsquedas bibliográficas, eliminación de duplicados, selección de trabajos y seguimiento de referencias. Para examinar ataques y defensas incorporan también documentación técnica. La clasificación relaciona algoritmos con sus supuestos de funcionamiento y muestra que las defensas dependen del adversario considerado. Una caída y una intervención deliberada pueden afectar la coordinación de formas diferentes. El estudio reúne mecanismos de protección y discute sus costes. Sus aportes consisten en una taxonomía y una síntesis de trabajos previos sobre amenazas, vulnerabilidades y medidas de defensa.

Dato concreto y localización

La búsqueda sobre algoritmos de consenso culminó en 126 artículos de investigación y nueve algoritmos. El recuento aparece en la sección II.A.1, p. 143634 (p. 4 del PDF).

Nota personal y uso en TA-IND-01

La revisión aporta un vocabulario para distinguir caídas, problemas de comunicación y conductas maliciosas. En la comparación permite delimitar qué fallos se analizan y examinar por separado cómo se coordina una operación y cómo se recupera una réplica. La documentación de Cassandra y MongoDB deberá respaldar la atribución de mecanismos concretos a cada operación.

Ficha 3. Arranque en frío en plataformas serverless

Referencia IEEE

M. Golec, G. K. Walia, M. Kumar, F. Cuadrado, S. S. Gill, and S. Uhlig, «Cold start latency in serverless computing: A systematic review, taxonomy, and future directions», *ACM Computing Surveys*, vol. 57, no. 3, Nov. 2024, Art. no. 65. DOI: 10.1145/3700875

DOI

10.1145/3700875

Resumen

Una plataforma serverless puede liberar recursos cuando disminuye la demanda. Si llega una solicitud y el entorno de ejecución debe prepararse desde cero, el usuario puede percibir una demora adicional. El artículo revisa estudios sobre las causas del arranque en frío y las estrategias para mitigarlo. Buscan trabajos en bibliotecas digitales, filtran los resultados y amplían la selección siguiendo referencias. La taxonomía agrupa enfoques como la caché, la optimización de aplicaciones y el aprendizaje automático. La revisión expone compromisos entre latencia, aprovisionamiento y coste, y reúne preguntas abiertas sobre la comparación de soluciones y la calidad del servicio.

Dato concreto y localización

La selección inicial dejó 25 trabajos y el rastreo retrospectivo de referencias elevó el conjunto a 32. Los autores describen el proceso en la sección 4.1, p. 65:13 (p. 13 del PDF).

Nota personal y uso en TA-IND-01

La revisión ofrece una razón metodológica para separar los efectos de inicialización de las mediciones en régimen estable. En el laboratorio de Cassandra y MongoDB esto se traduce en registrar cuándo se inicia el cliente, documentar el calentamiento si se utiliza y comparar observaciones en condiciones equivalentes. Su aporte se limita al diseño de la medición.

Ficha 4. Consistencia y rendimiento en bases NoSQL

Referencia IEEE

S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo, and E. Andrade, «Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications», *Journal of Cloud Computing*, vol. 13, no. 1, Nov. 27, 2024, Art. no. 158. DOI: 10.1186/s13677-024-00716-7

DOI

10.1186/s13677-024-00716-7

Resumen

En el artículo se investiga cómo cambian el rendimiento y el tiempo de respuesta al modificar la consistencia en bases NoSQL replicadas. Prueban Cassandra, MongoDB y Redis en Google Cloud, con despliegues de una y varias regiones. Generan carga con Locust y comparan configuraciones que los autores denominan débiles y fuertes. El impacto depende del motor, la operación y la ubicación de las réplicas. El trabajo identifica diferencias estadísticamente significativas entre configuraciones en los escenarios multirregionales evaluados. La comparación permite estudiar cuánto cuesta coordinar las operaciones en condiciones concretas y por qué los resultados deben interpretarse junto con la configuración.

Dato concreto y localización

El estudio ejecutó 200 escenarios: 80 de MongoDB, 80 de Cassandra y 40 de Redis. Cada escenario duró 300 segundos. Véase «Input parameters and evaluated metrics», p. 7-8, antes del listado 1.

Nota personal y uso en TA-IND-01

Es la fuente principal para relacionar la configuración de consistencia con el rendimiento observado en ambos motores. Sus escenarios también orientan el diseño del experimento propio. Las mediciones publicadas permiten discutir el coste de la coordinación bajo las configuraciones estudiadas. El comportamiento ante una partición y las garantías exactas de cada modo requieren evidencia adicional.

Ficha 5. Telemetría distribuida de microservicios

Referencia IEEE

M. Otero, J. M. García, and P. Fernandez, «An extensible lightweight framework for distributed telemetry of microservices», *Sustainable Computing: Informatics and Systems*, vol. 46, 2025, Art. no. 101100. DOI: 10.1016/j.suscom.2025.101100

DOI

10.1016/j.suscom.2025.101100

Resumen

En el artículo se estudia cómo facilitar el diagnóstico de microservicios sin exigir una configuración compleja de la instrumentación. Proponen un componente para Node.js que aprovecha descripciones OpenAPI, registra telemetría de forma distribuida y admite extensiones. Evalúan el prototipo con una carga sintética, un microservicio de Bluejay y una comparación con Jaeger. Los resultados muestran que el coste de instrumentar depende de la carga y de la métrica observada. En algunos escenarios el consumo adicional de memoria es apreciable. Los autores discuten la posibilidad de ajustar la recopilación según las necesidades del servicio y presentan estimaciones energéticas mediante un modelo.

Dato concreto y localización

En el escenario sintético ONES, la memoria pasó de 21,58 MB sin telemetría a 40,99 MB con telemetría. En HUNDREDS pasó de 138,63 a 140,23 MB. Véase la sección 4.2.1, tabla 1, p. 10.

Nota personal y uso en TA-IND-01

El estudio aporta un ejemplo de cómo el mecanismo de observación puede afectar la medición. Para el laboratorio justifica documentar qué registros y métricas están habilitados y mantenerlos comparables entre corridas. Las cifras corresponden al prototipo evaluado. Su uso en el informe se limita al criterio metodológico de controlar el coste de instrumentación.